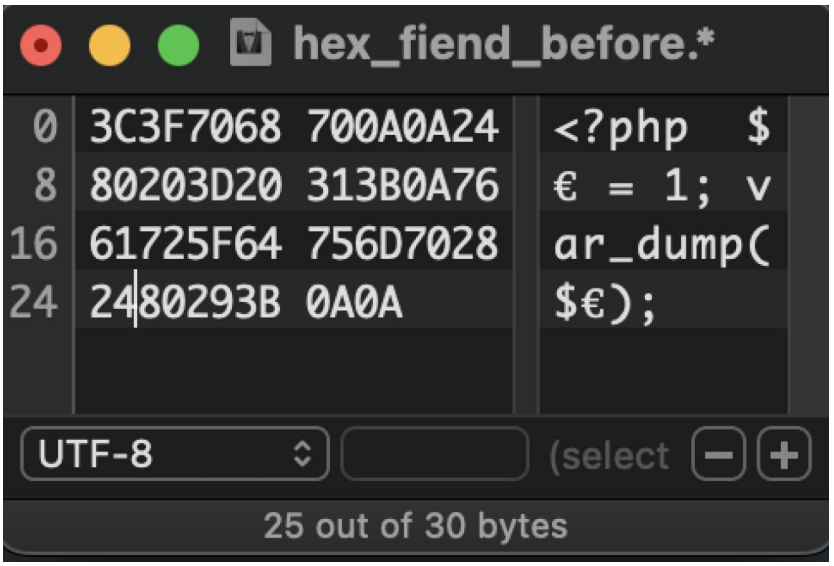


Figure 8-2. *hex edit raw byte values after…*

Make sure to save the changes. What we've done here is swap the `x` character of `$x` for a different byte value: *0x80*. This is an invalid sequence in UTF-8 (*0x80* is a continuation byte, and the *0x24* byte before it is not a valid leading byte), and extended ASCII encodings (ISO-8859-*) do not use the range *0x80–0x9F* for printable characters. The specification for UTF-8 states that invalid sequences should be handled gracefully by clients, so attempting to view `foo.php` in an editor or shell might generate warnings, but it should show a placeholder character where the `x` used to be.

```
$ cat foo.php
<?php

$? = 1;
var_dump($?);
```

My trusty Mac/PHPStorm combo renders a question mark placeholder, but yours will very likely be something different, perhaps a numeric placeholder. Those of you following along at home might get something like `$<80> = 1; var_dump($<80>);` – it will vary between systems and their age. The big question now is will it execute? It does, because we haven't broken any of PHP's rules on variable name symbols.

```
$ php foo.php
int(1)
```